

Statistiques

Centrer les données

```
temps = np.array([15.2, 15.5, 15.1, 15.3, 15.4,  
                  25.8, 26.1, 25.9,  
                  15.2, 15.3, 15.1])
```

1. Centrez les données

Méthode: retirer la moyenne des températures aux températures de `temps`.

```
# TODO
```

Questions

1. Que représente maintenant une valeur positive ? Une valeur négative ?
 2. Trouvez les températures basses et hautes ?
-

Jeu de données très simple

On prend deux valeurs :

```
x = [8, 12]
```

Centrer

On enlève la moyenne.

La moyenne vaut :

$$\mu = \frac{8 + 12}{2} = 10$$

Données centrées :

$$x_c = [8 - 10, 12 - 10] = [-2, 2]$$

Centrer = déplacer les données pour les ramener autour de 0. Mais **l'écart entre les valeurs reste le même** (4 unités).

Réduire

On divise par l'écart-type :

Écart-type des valeurs [8, 12] :

$$\sigma = 2$$

Réduction :

$$x_s = \frac{x_c}{\sigma} = [-2/2, ; +2/2] = [-1, ; +1]$$

Interprétation intuitive

Après réduction, les données deviennent :

[-1, +1]

Pourquoi est-ce intéressant ?

1. La **moyenne** devient **0**
2. L'**écart-type** devient **1**
3. Les valeurs ne s'expriment plus "en unités (km/h, kg, degrés...)" mais en **écarts-types par rapport à la moyenne**

Donc :

1. -1 = *une valeur située un écart-type sous la moyenne*
2. +1 = *une valeur située un écart-type au-dessus de la moyenne*

On ne regarde plus l'unité, mais la position relative.

Ce que la réduction "corrige" que le centrage seul ne corrige pas

Exemple :

Données A : x = [50, 60] Données B : y = [1, 2]

Si on les centre :

$$x_c = [-5, +5] \quad y_c = [-0.5, +0.5]$$

On ne peut **toujours pas** les comparer : les deux variables ont des amplitudes différentes (10 vs 1).

La réduction met les deux variables sur la même échelle, avec un écart-type de 1.

Résumé pédagogique

1. **Centrer** → enlever la moyenne → ramener les données autour de 0.
2. **Réduire** → diviser par l'écart-type → mettre les données sur une échelle comparable.
3. Après centrage-réduction :

moyenne = 0 écart-type = 1 les valeurs expriment *combien d'écarts-types elles s'éloignent de la moyenne*.

Comprendre l'intérêt du centrage-réduction

On dispose de deux jeux de données mesurés sur les mêmes individus :

1. **Température corporelle (°C)**
2. **Fréquence cardiaque (bpm)**

```
import numpy as np
```

```
temp = np.array([36.8, 37.1, 36.9, 37.5, 38.2, 36.7, 37.0])  
heart = np.array([62, 70, 65, 74, 90, 60, 68])
```

Grandeurs

Les deux grandeurs ne sont **pas du tout** sur la même échelle :

1. la température varie autour de **37°C**,
2. la fréquence cardiaque varie autour de **65–80 bpm**.

Donc on ne peut pas les comparer directement.

Centrez les données

```
# TODO
```

Questions

1. Que représente maintenant une valeur positive ?
 2. Et une valeur négative ?
-

Réduire normalisation de l'écart-type

On divise chaque série centrée par son écart-type, utilisez `np.std`

#TODO temp_scaled et heart_scaled

Pourquoi est-il maintenant pertinent de comparer `temp_scaled` et `heart_scaled` ?

Identifier dans chaque série normalisée :

1. les valeurs proches de **0**
2. les valeurs supérieures à **+1**
3. les valeurs inférieures à **-1**

Que peut-on conclure ?

Résumé de la méthode

Les données centrées-réduites ont :

1. **moyenne = 0**
2. **écart-type = 1**
3. les valeurs s'expriment en **écarts-types par rapport à la moyenne**

Ainsi :

1. **+2** signifie : *2 écarts-types au-dessus de la moyenne* → valeur **très atypique**
2. **-1** signifie : *1 écart-type en dessous de la moyenne* → valeur **un peu faible**
3. **0** signifie : *dans la moyenne*

Les deux variables sont maintenant exprimées **sur la même échelle**, donc comparables.

ACP

L'ACP = *Analyse en Composantes Principales*

Objectif : **résumer un jeu de données** tout en gardant l'essentiel de l'information.

L'ACP ne travaille pas sur des matrices forcément carrées, mais sur des matrices de dimensions **n**x**p**

n = nombre d'individus / observations **p** = nombre de variables explicatives

ACP 1/2

1. On a souvent plusieurs variables → taille, poids, âge...
 2. Elles sont liées → les gens plus grands pèsent souvent plus.
 3. L'ACP cherche **les directions où les données varient le plus**.
 4. Ces directions s'appellent **les composantes principales** (PC1, PC2...).
 5. On commence par **centrer les données** pour éliminer l'effet du niveau moyen.
 6. On calcule ensuite la **matrice de covariance** qui mesure comment les variables varient ensemble.
-

ACP 2/2

7. Les **vecteurs propres** de cette matrice donnent les directions principales.
 8. Les **valeurs propres** indiquent l'importance de chaque direction (PC1 explique le plus).
 9. On **projette les données** sur ces directions pour obtenir de nouvelles coordonnées (PC1, PC2).
 10. Cela permet de **résumer l'information** avec moins de dimensions (souvent 2 au lieu de plusieurs).
 11. On peut alors **visualiser, comparer, ou détecter des tendances** beaucoup plus facilement.
-

Numpy projeter les données sur PC1 et PC2

Xc matrice centrée eigvecs vecteurs propres

X_proj = Xc @ eigvecs

ACP : réécrire les données dans les directions les plus importantes.

En résumé :

L'ACP tourne le nuage de points pour le regarder selon les directions qui expliquent le plus de variation. L'ACP garde les individus mais remplace les variables d'origine par 2 nouvelles variables (PC1 et PC2) qui résument presque toute l'information.

3 variables : taille, poids, âge

Les données

```
import numpy as np

# Données : [taille (cm), poids (kg), âge (ans)]
X = np.array([
    [170, 65, 25],
    [180, 80, 32],
    [160, 55, 22],
    [175, 75, 29],
    [165, 60, 24]
], dtype=float)
```

Centrer les données

```
X_mean = X.mean(axis=0)
X_centered = X - X_mean
```

Matrice de covariance

```
Cov = np.cov(X_centered, rowvar=False)
```

Cette matrice indique comment les variables varient ensemble.

Cov(taille, poids) → grande valeur positive
Cov(taille, âge) → valeur positive modérée
Cov(poids, âge) → valeur positive modérée

Calcul valeurs propres / vecteurs propres

```
vals, P = np.linalg.eig(C)
```

PC1 = la direction de variation la plus importante
PC2 = la direction secondaire
PC3 = la direction la moins importante

Projection des données sur les composantes principales

```
# Tri par importance décroissante
idx = np.argsort(vals)[::-1]
vals = vals[idx]
P = P[:, idx]

# Projection
X_proj = Xc @ P
```

Synthèse

1. PC1 sépare majoritairement les personnes selon taille + poids (souvent très corrélés, la principale source de variation).
 2. PC2 capture ce qui varie indépendamment de PC1 (par exemple l'âge si moins corrélé aux deux autres).
 3. PC3 explique très peu → bruit résiduel.
-

Les nouvelles coordonnées X_proj sont :

1. PC1 → résumé principal de la morphologie
 2. PC2 → variation secondaire
 3. PC3 → très peu d'information utile
-

Exercice corrigé

Vitesse, force, endurance, agilité.

Utilisez l'APC

```
import numpy as np

X = np.array([
    [28, 110, 42, 70],
    [26, 105, 38, 65],
    [30, 120, 46, 72],
    [24, 100, 35, 60],
    [29, 115, 44, 68],
    [27, 108, 40, 66],
```

```
[31, 125, 48, 75],  
 [25, 102, 36, 62]  
], dtype=float)
```

Centrer réduire

```
# 1) Centrer  
Xc = X - X.mean(axis=0)  
  
# 2) Réduire  
Xs = Xc / X.std(axis=0)
```

Matrice de covariance

```
Cov = np.cov(Xs, rowvar=False)  
vals, vecs = np.linalg.eig(Cov)  
  
# Trier par importance décroissante  
idx = np.argsort(vals)[::-1]  
vals = vals[idx]  
vecs = vecs[:, idx]
```

Cercle de corrélation

```
corr = vecs * np.sqrt(vals)  
corr_circle = corr[:, :2]  
print(corr_circle)
```

Interprétation simple pour les étudiants

Chaque ligne correspond à une variable (vitesse, force, endurance, agilité). Chaque colonne correspond à PC1 et PC2. Une valeur proche de +1 ou -1 = variable fortement liée à la composante. Une valeur proche de 0 = variable peu liée à cette composante. On place ces points sur un cercle de rayon = 1. Si deux variables sont proches dans le cercle → fortement corrélées entre elles.

En résumé

PC1 capture surtout les variables qui ont une grande coordonnée dans la colonne PC1. PC2 capture celles alignées verticalement dans la colonne PC2. Les variables proches en angle $\rightarrow$ corrélées.

Représentation graphique

Résumé

1. Vitesse, Force, Endurance sont fortement corrélées entre elles et alignées sur PC1.
 2. Agilité suit globalement la même direction mais avec une composante PC2 plus négative.
 3. Toutes les variables étant proches du cercle, elles sont bien expliquées par les deux premières composantes principales.
-

Code complet Numpy

```
import numpy as np
import matplotlib.pyplot as plt

# -----
# 1. Données
# -----
X = np.array([
    [28, 110, 42, 70],
    [26, 105, 38, 65],
    [30, 120, 46, 72],
    [24, 100, 35, 60],
    [29, 115, 44, 68],
    [27, 108, 40, 66],
    [31, 125, 48, 75],
    [25, 102, 36, 62]
], dtype=float)

variables = ["Vitesse", "Force", "Endurance", "Agilité"]

# -----
# 2. Centrer et réduire
# -----
```

```

Xc = X - X.mean(axis=0)
Xs = Xc / X.std(axis=0)

# -----
# 3. Matrice de covariance + ACP
# -----
Cov = np.cov(Xs, rowvar=False)
vals, vecs = np.linalg.eig(Cov)

# tri décroissant
idx = np.argsort(vals)[::-1]
vals = vals[idx]
vecs = vecs[:, idx]

# -----
# 4. Cercle de corrélation : PC1 & PC2
# -----
corr = vecs * np.sqrt(vals)      # corrélations variables <-> axes
corr_circle = corr[:, :2]        # coordonnées pour le cercle

# -----
# 5. Tracé du cercle de corrélation
# -----
fig, ax = plt.subplots(figsize=(6, 6))

# cercle unité
theta = np.linspace(0, 2*np.pi, 300)
ax.plot(np.cos(theta), np.sin(theta), color="orange")

# axes principaux
ax.axhline(0, color="orange")
ax.axvline(0, color="orange")

# flèches des variables
for i, var in enumerate(variables):
    x, y = corr_circle[i]
    ax.arrow(0, 0, x, y, head_width=0.03, length_includes_head=True)
    ax.text(x * 1.08, y * 1.08, var)

ax.set_title("Cercle de corrélation (PC1-PC2)")
ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_aspect("equal")

plt.grid(False)
plt.show()

```

TP

Dynamique des espèces